Autonomous Newsroom

Confer Inc. · AI/ML Engineering Take-Home

The product is a small news page that updates itself. The real test is the team of OpenClaw and Hermes agents that builds and ships it, and how well you run that team.

Graded on agent delivery, not code

~5 days, sooner is better

OpenClaw + Hermes Agent

Free stack + Terraform

Hand over: link + repo + DELIVERY.md

What this tests (read this first)

We are not grading your code. We are grading how you get a team of autonomous agents to deliver it.

At Confer, client software is delivered by teams of autonomous agents running on OpenClaw and Hermes Agent. A manager or orchestrator hands out tickets; planners, coders, reviewers and testers pick them up and pass work along, much like a human team. The engineer's job is to:

turn a business need into a sound technical plan;

set the agents up and keep them productive;

own what they ship, including knowing what went wrong and why.

This take-home is that job in miniature. The product below is small on purpose: it is the excuse. The agent team that builds it is the test.

Plan as much as you like with any strong model (Claude, ChatGPT, Gemini…), and use AI to help you set up the agents. Every tool is allowed.

The agents write the code, not you. Do not hand-write the feature code, and do not paste a line-by-line plan into a weaker model. It is fine if your agents make mistakes or push messy code. Show us how your system catches them.

We want to see how you set the agents up, how tickets flowed, which models did what, where the tokens went, and what you did when something broke.

The product needs to be clickable, and only a bit of it has to work.

The product your agents build

A small newsroom that runs itself. Pick a topic (suggested: "latest AI/ML news", but any beat is fine). A public page lists recent articles on that topic and refreshes on a schedule, with nobody pressing a button.

The core is deliberately small. This is all we require:

a public page listing recent articles on your topic;

each article shows a one-line summary, a topic tag and a source link;

the list is refreshed on a schedule, and the articles are saved in a database (Postgres on your VM or SQLite; your call);

the page shows "updated X ago".

Use real, public news from a source that needs no paid key, for example a Google News RSS search feed or the Hacker News API. No personal data.

Example: what a great submission looks like. We open your URL and see about 10 recent articles on your topic, each with a one-line summary, a tag and a source link, and a footer reading "updated 2h ago." Behind it, a scheduled job pulls a public feed every few hours, a model summarizes and tags each item, the rows land in an articles table, and the page reads them. Behind that, in your repo, we see a ticket for each piece, a commit or PR from the agent that did the work, and at least one review that sent something back.

If your agents have time

dedupe near-identical stories;

topic filters (click a tag to narrow the list);

more than one source;

a search box;

a small page showing the last few refresh runs.

None of these is needed for a good score. A tidy core with clean handoffs beats a bigger product with a messy trail.

Suggested team for this product

Role	Job on this product	Example tickets or checks
Orchestrator	Turns this brief into tickets, assigns them, tracks status, and re-assigns work when review or test sends it back.	The ticket list itself.
Builder (one or more)	Picks up a ticket and opens a PR for it.	"Scaffold DB schema" · "RSS collector" · "Summarizer with fallback model" · "UI list page" · "Deploy"
Reviewer	Reads each PR against its ticket and sends back anything that misses it.	For example, reject a summarizer that has no fallback when the model answers 429.
Tester	Runs the thing instead of only reading it.	"Collector writes rows" · "Page loads and shows 'updated X ago'"

This is an example, not a requirement. Design a different team if you think it will deliver better (two builders in parallel, a separate planner, a docs agent). It still has to meet the team rules below, and DELIVERY.md should say why you split the work the way you did.

Two things run here, and they are different: the agents that build the product, and the product itself. The product's own scheduled job may call the free models directly from ordinary code. Running the collector as an OpenClaw or Hermes agent too (both runtimes have a built-in scheduler) is a plus, not a requirement.

Keep the summarizer cheap. The free model allowance is small, so summarize each article once, store the result, and fall back to a second free model if the first one refuses (HTTP 402 or 429).

Confer runs information-gathering agents like this for its own team, so this is a small version of real work.

Use AI tools, that's the point. Plan with the strongest model you like and let it help you set up your agents. Blocked? Tell us.

Your agent delivery team (hard rules)

Runtimes: OpenClaw and Hermes Agent, both open source. At least one agent runs on each. Together they form a delivery team with at least these roles:

orchestrator or manager: breaks the work into tickets and assigns them;

builder or coder;

reviewer;

tester.

One agent may hold two roles if your write-up says why.

Tickets flow between agents through something we can inspect: GitHub Issues and PRs, a queue table, or ticket files. The orchestrator assigns, agents pick up and hand off, and the review and test steps can send work back.

The agents write the code. The git history must show agent-authored commits or PRs, for example a distinct author per agent or co-author trailers. You review and merge.

Free stack only (see the box below): a free Linux VM provisioned with Terraform (or OpenTofu), free models, free hosting. No paid keys, and no real or personal data.

It runs: a public URL we can open, which loads and does the core thing. Partial is fine.

The free stack (checked October 2026)

Layer	Default (free)	Notes
Server	Oracle Cloud Always Free: one Arm VM.Standard.A1.Flex with 2 OCPU / 12 GB RAM, Ubuntu 24.04 aarch64, in your home region	The only always-free VM with enough RAM for OpenClaw, Hermes and Postgres. Oracle cut the Arm allowance from 4/24 to 2/12 in mid-2026, and many guides are stale. Stay within 2/12. A card is needed at sign-up for identity checks; nothing is charged within Always Free.
Provisioning	Terraform (free) or OpenTofu with the oracle/oci provider. Start from Oracle's examples/always_free, trimmed to one instance with no load balancer and no DB.	Keep the .tf files in infra/. Use local state, and git-ignore state files and keys, which hold secrets. A1 needs an aarch64 image and a shape_config.
Agent runtimes	OpenClaw (Node 24.16+ or 26.1+) and Hermes Agent (Nous Research), both installed with their install.sh	OpenClaw: openclaw agents add for multiple agents. Hermes: profiles (one per agent) and delegate_task. Keep both dashboards on localhost and reach them over an SSH tunnel.
Models	Ollama Cloud free plan (OLLAMA_API_KEY, OpenAI-compatible base https://ollama.com/v1); both runtimes have an ollama-cloud provider	Since 31 Aug 2026 the free plan is a small, unpublished monthly allowance with 1 concurrent request. Make the model configurable, add a fallback (OpenRouter :free models, the Gemini API free tier, or Groq) and handle HTTP 402/429.
UI hosting	Serve it from the VM (Caddy or nginx on 80/443), or use a cloudflared Quick Tunnel for a free trycloudflare.com URL with no account or domain	Cloudflare Pages (free) works for a static front end. Vercel Hobby is allowed but is "non-commercial personal use only".
Token tracking	OpenClaw /usage cost and its OpenTelemetry openclaw.tokens metric (tagged per agent); Hermes hermes insights or the dashboard Analytics page; the Ollama usage page	Free extras: one OpenRouter key per agent gives per-agent attribution; Langfuse Cloud Hobby is free with no card. Don't self-host Langfuse on the 12 GB VM.

If Oracle says "Out of host capacity": try another availability domain, don't pin a fault domain, try 1 OCPU / 6 GB, or retry later. If it is still blocked after a day, use plan B: the AWS Free plan (credits for 6 months; m7i-flex.large, 8 GiB). The emergency fallback is your own machine (8 GB+ RAM) plus a Cloudflare Quick Tunnel; say so in DELIVERY.md. Oracle may reclaim an Always Free VM that sits idle for 7 days, so keep a light cron running. Blocked on free compute? Reply to our email; we'll help.

What you hand over

Reply to our email with:

Live UI link: served from your VM, a Cloudflare Tunnel, your own domain, or Vercel.

GitHub repo: public, or private with Shrikanth-Confer invited as a collaborator. It must contain:

the product code, written by your agents;

infra/ with your Terraform;

agents/ with each agent's OpenClaw or Hermes config, prompts and skills (secrets removed);

the ticket trail.

DELIVERY.md in the repo, covering:

the agent roster: name, runtime, role, model and tools for each agent;

a diagram of how a ticket moves from idea to merged code;

what each agent actually did;

one thing that broke and how it was caught or fixed;

what you would change.

Token & model report (in DELIVERY.md or linked): tokens per agent and per model, split into planning, coding, review/test and retries, and in plain words where the tokens went. Estimates are fine if you say so.

Trace excerpts: a few real log lines showing a handoff, and a review rejection or retry.

Reproduce: how to stand it all up again, for example terraform apply followed by one install script.

Optional, and valued: a 3–5 minute screen recording of your agent team at work.

How we score it (100 points)

Area	Points	What earns it
Agent team design & orchestration	30	Clear roles; the orchestrator really assigns the work; handoffs are explicit; the review and test gates actually block bad work.
Evidence the agents delivered	25	Agent-authored commits or PRs, a real ticket trail, traces of handoffs, and a rejection or retry handled.
Models, tokens & observability	15	A sensible model for each role (strong for planning, cheaper for routine work), and a token report that explains where the tokens went.
Infra as code & reproducibility	15	Terraform creates the VM, secrets stay out of git, and someone else could rerun it.
It runs	10	The public link loads and the core feature works. Partial is fine.
Judgement & write-up	5	DELIVERY.md is honest about what broke and what you'd change.

Not passing: the code was written by hand or copy-pasted rather than delivered by agents, there is no public link, or there is no repo. In the follow-up call we'll ask you to walk us through your agent team and have it make a small change live.

Timeline & help

Aim to send it within 5 calendar days of receiving this brief; our email gives the exact date. Sooner is better: we move quickly and roles don't stay open long.

Need more time? Tell us before the deadline.

Blocked on free hosting, models or setup? Reply to our email. We would rather unblock you than have you stuck: working out what to build is the test, not finding resources.

Keep your link up until we've reviewed it, which is about a week.

Resources

OpenClaw: docs · repo · multi-agent · Ollama Cloud provider · usage tracking.

Hermes Agent: docs · repo · profiles · delegation · cron.

Server & IaC: Oracle Always Free · OCI Terraform always_free example · Terraform · OpenTofu · AWS Free plan (plan B).

Models: Ollama Cloud (API keys) · OpenRouter free limits · Gemini API free tier · Groq limits.

Hosting & tracking: Cloudflare Quick Tunnel · Cloudflare Pages · Langfuse Hobby.

Confer Inc. · AI/ML Engineering Take-Home · free tiers, synthetic or public data only · graded on agent delivery, not code.